Plan de Mémoire de Fin d'Études en Génie Logiciel

Voir Guide d'élaboration du rapport du PFE (Guide PFE.pdf)

Page de Garde

Voir Formulaire de stages (https://www.episup.com/fr/stages)

Résumé

- *Résumé du projet en 150 à 300 mots, incluant les objectifs, la méthodologie, les résultats obtenus et les contributions majeures.*
- *Mots-clés : 5 à 7 mots-clés pertinents pour le projet.*

Dédicaces (Optionnel)

Dédicaces personnelles.

Remerciements

Remerciements aux personnes et institutions ayant contribué à la réalisation du projet.

Table des Matières

Liste des chapitres, sections et sous-sections avec numéros de page.

Liste des Figures

Liste des figures avec leurs titres et numéros de page.

Liste des Tableaux

Liste des tableaux avec leurs titres et numéros de page.

Liste des Abréviations

Liste des abréviations utilisées dans le mémoire avec leurs significations.

Introduction Générale

- Contexte et cadre d'étude
- Problématique
- Objectifs du mémoire
- Approche méthodologique
- Plan du mémoire

CHAPITRE 1 : CONTEXTE ET CADRE DU PROJET

Introduction

1.1. Présentation de la Société d'Accueil

1.1.1. Structure organisationnelle

1.1.2. Domaine d'activité

1.2. Fondements du projet

1.2.1. Problématique

1.2.2. Objectifs du projet

1.2.3. Modèle de développement

Le modèle de développement définit comment les différentes phases du projet (analyse, conception, implémentation, tests, déploiement) sont organisées et gérées :

- *Modèle en Cascade (Waterfall) : Approche linéaire où chaque phase doit être terminée avant de passer à la suivante.*
  Exemple : Analyse → Conception → Implémentation → Tests → Déploiement.
- *Modèle Itératif et Incrémental : Le projet est divisé en itérations, chaque itération produisant un incrément du système.*
  Exemple : Développement par cycles courts avec des livraisons fréquentes.
- *Modèle Agile : Approche flexible et collaborative, souvent basée sur des frameworks comme Scrum ou Kanban.*
  Exemple : Développement en sprints de 2 à 4 semaines.
- *Modèle en V : Variante du modèle en cascade où les phases de test sont alignées avec les phases de développement.*
  Exemple : Conception → Tests unitaires → Implémentation → Tests d'intégration.

1.3. État de l'art

1.3.1. Collecte de l'information (si nécessaire !)

1.3.2. Prospection de solutions existantes

1.3.3. Synthèse critique et positionnement

Conclusion

CHAPITRE 2 : ANALYSE DES BESOINS ET SPÉCIFICATIONS

Introduction

2.1. Vision du Projet

2.1.1. Exigences fonctionnelles

2.1.2. Exigences non fonctionnelles

Elles concernent les aspects qualitatifs du système, tels que la performance, la sécurité, la disponibilité, l'évolutivité, la compatibilité, etc.

2.1.3. Prototype conceptuel et ergonomique (optionnel)

2.2. Spécification des exigences

2.2.1. Caractérisation des acteurs (fiches personas)

2.2.2. Identification des cas d'utilisation

2.2.3. Diagrammes des cas d'utilisation

2.2.4. Scénarios Utilisateur

Décrire les parcours typiques des utilisateurs dans le système, en mettant en avant les interactions entre les utilisateurs et le système : Scénarios nominaux, alternatifs et d'erreur(seulement pour les cas d'utilisation majeurs)

2.2.5. Diagrammes de séquence système

2.2.6. Contraintes techniques (optionnel)

Identifier les limitations qui doivent être prises en compte lors de la conception et de la mise en œuvre du système :

- *Contraintes Matérielles : Limitations liées au matériel (ex : puissance de calcul, mémoire, stockage).*
- *Contraintes Logicielles : Limitations liées aux logiciels ou aux technologies utilisées (ex : compatibilité avec des systèmes existants).*

- *Contraintes de Performance : Exigences en termes de temps de réponse, de débit, ou de disponibilité.*
- *Contraintes de Sécurité : Exigences en matière de protection des données et de confidentialité.*

2.3. Optimisation du développement

2.3.1. Backlog du Produit

2.3.2. Priorisation des fonctionnalités

Déterminer l'ordre dans lequel les fonctionnalités doivent être développées, en fonction de leur importance, de leur urgence et des ressources disponibles :

Méthodes de Priorisation :

- *MoSCoW : Classer les fonctionnalités en quatre catégories :*
  - *Must Have : Fonctionnalités essentielles sans lesquelles le projet ne peut pas être livré.*
  - *Should Have : Fonctionnalités importantes mais pas critiques.*
  - *Could Have : Fonctionnalités souhaitables mais non essentielles.*
  - *Won't Have : Fonctionnalités qui ne seront pas développées dans cette version.*
- *Matrice d'Eisenhower : Classer les fonctionnalités en fonction de leur urgence et de leur importance.*
- *Valeur vs Complexité : Évaluer chaque fonctionnalité en fonction de la valeur qu'elle apporte à l'utilisateur et de la complexité de sa mise en œuvre.*

2.3.2. Planification des Sprints

2.3.3. Architecture fonctionnelle du système (Site Map)

2.4. Validation des Besoins (si nécessaire ! Processus de développement en V)

Décrire le déroulement de la validation des besoins afin de s'assurer que les exigences sont bien comprises et acceptées par toutes les parties prenantes, ce qui réduit les risques de malentendus ou de changements ultérieurs

Participants :

Les ateliers impliquent généralement des représentants des utilisateurs finaux, des experts métier, des développeurs, et des chefs de projet.

Déroulement :

- *Présentation des exigences : Présenter les exigences fonctionnelles et non fonctionnelles aux participants.*
- *Discussion et Feedback : Recueillir les retours des participants sur les exigences proposées.*
- *Validation : Obtenir un accord formel sur les exigences validées.*

Conclusion

CHAPITRE 3 : CONCEPTION ET MODÉLISATION DU SYSTÈME

Introduction

3.1. Conception Objet

3.1.1. Diagrammes de classes participantes

3.1.2. Diagrammes de classes de conception

Représenter la structure détaillée des classes, y compris les attributs, les méthodes, et les relations techniques

3.1.3. Diagramme de classes de Persistance

Représenter les classes qui interagissent avec la base de données, y compris les entités persistantes et leurs relations

3.1.4. Dictionnaire des attributs

3.2. Modélisation des Interactions

Diagrammes de Séquence de conception des cas d'utilisation majeurs

3.2.1. Cas 1 : …

3.2.2. Cas 2 : …

3.3. Modélisation Relationnelle

3.3.1. Conception de la Base de Données

3.3.2. Scripts SQL

3.4. Autres Modèles UML (optionnel)

3.4.1. Diagrammes d'activités

3.4.2. Diagrammes d'états-transitions

3.4.3. Diagramme de composants

3.4.4. Diagramme de déploiement

Conclusion

CHAPITRE 4 : MISE EN ŒUVRE TECHNIQUE ET DEVOPS

Introduction

4.1. Choix Technologiques

4.1.1. Architecture logicielle

a. *Paradigme architectural*

Décrire la structure globale du système, en définissant les composants principaux, leurs responsabilités, et leurs interactions. Elle Permet de concevoir un système robuste, évolutif, et facile à maintenir :

- *Architecture Monolithique : Le système est conçu comme une seule unité, avec tous les composants étroitement couplés.*
- *Architecture en Couches (Layered Architecture) : Le système est divisé en couches (ex : présentation, logique métier, accès aux données).*
- *Architecture Microservices : Le système est divisé en petits services indépendants, chacun ayant sa propre base de données et son propre cycle de vie.*
- *Architecture Orientée Services (SOA) : Le système est composé de services réutilisables qui communiquent via des protocoles standard (ex : SOAP, REST)*

b. *Modèle de conception architecturale (MVC, MVVM, MVP, …)*

4.1.2. Environnement technique

Décrire les outils, les technologies, et les infrastructures utilisés pour développer, tester, et déployer le système. L'environnement technique inclut les langages de programmation, les frameworks, les bases de données, les outils de développement, et les plateformes de déploiement :

a. *Langages de Programmation*

b. *Frameworks et Bibliothèques (Front-end & Back-end)*

c. *Bases de Données (Relationnelle, NoSQL, …)*

d. *Outils de Développement (IDE, Gestion de Code Source & Gestion de Projet)*

e. *Plateformes de Déploiement (Cloud, Conteneurs & Serveurs)*

f. *Outils de Tests (Tests Unitaires, Tests d'Intégration & Tests de Performance)*

4.2. Intégration des Pratiques DevOps

4.2.1. Contrôle du Code Source avec DevOps

Gérer les versions du code source de manière collaborative et efficace, en intégrant les pratiques DevOps (Branches et Workflows, Intégration Continue, …) pour améliorer la productivité et la qualité du code

4.2.2. Design Patterns et DevOps

Présenter les modèles de conception couramment utilisés permettant de structurer le code de manière à faciliter l'intégration des pratiques DevOps, comme la testabilité, la modularité, et la déployabilité :

- *Singleton : Assure qu'une classe n'a qu'une seule instance, utile pour gérer les configurations ou les connexions à la base de données.*
- *Factory : Permet de créer des objets sans spécifier la classe exacte, utile pour la modularité.*

- *Observer : Permet à des objets de s'abonner et de réagir aux changements d'état d'un autre objet, utile pour les systèmes événementiels et la modularité.*
- *Dependency Injection : Facilite la gestion des dépendances et améliore la testabilité (tests unitaires et d'intégration).*

4.2.3. Tests Automatisés dans DevOps

Automatiser les tests pour garantir la qualité du code à chaque étape du pipeline DevOps, de l'intégration continue au déploiement :

- *Tests Unitaires : Testent des unités individuelles de code (JUnit (Java), pytest (Python), NUnit (.NET)).*
- *Tests d'Intégration : Testent l'interaction entre plusieurs composants ou services (Selenium, Postman, RestAssured).*
- *Tests de Performance : Testent la performance du système sous charge (JMeter, Gatling).*
- *Tests de Sécurité : Testent les vulnérabilités du système (OWASP ZAP, SonarQube)*

4.2.4. CI/CD et Déploiement Continu

Mettre en place un pipeline d'intégration continue (CI) et de déploiement continu (CD) pour automatiser la construction, les tests, et le déploiement du code :

- *Build : Compilation du code et création des artefacts (ex : fichiers JAR, Docker images).*
- *Tests : Exécution des tests unitaires, d'intégration, et de performance.*
- *Déploiement : Déploiement automatique sur les environnements cibles.*
- *Surveillance : Surveillance des performances et des erreurs en production.*

4.2.5. Cloud et Conteneurisation avec DevOps

Utiliser les technologies cloud et de conteneurisation pour améliorer la scalabilité, la flexibilité, et la gestion des applications dans une approche DevOps :

- *Cloud Computing : Utilisation de services comme AWS, Azure, ou Google Cloud Platform pour héberger les applications.*
- *Conteneurisation (Docker : Création de conteneurs pour encapsuler les applications et leurs dépendances + Orchestration : Utilisation de Kubernetes pour gérer les conteneurs à grande échelle).*

4.3. Développement sécurisé et confidentialité des données

4.3.1. Principes fondamentaux de la sécurité du développement

- *Contrôle des accès et authentification : gestion des utilisateurs, MFA.*
- *Validation et gestion des entrées utilisateur : protection contre les injections SQL, XSS, etc.*
- *Chiffrement des données : stockage et transmission sécurisés des données sensibles (ex. : AES, RSA).*
- *Gestion des vulnérabilités : mise à jour des dépendances, tests de pénétration et audits de code.*

4.3.2. Confidentialité des données et conformité réglementaire

- *Protection des données personnelles (principes du RGPD : Règlement Général sur la Protection des Données) : Limitation des finalités, Minimisation des données, Limitation de la conservation, …*
- *Stockage et traitement sécurisé : bases de données chiffrées, journalisation sans exposition d'informations sensibles.*
- *Gestion des droits et permissions (principe du moindre privilège PoLP : Principle of Least Privilege) : accorder aux utilisateurs uniquement les permissions strictement nécessaires pour accomplir leurs tâches, et rien de plus.*

4.4. Autres Parties Spécifiques au Sujet

4.5. Jeu d'Essais et Manuel d'utilisateur

Pour chaque fonctionnalité : Capture d'écran annotée + Procédure étape par étape + Exemple concret

Conclusion

Conclusion Générale

1. Récapitulation de la Démarche
2. Résultats Obtenus
3. Problèmes Rencontrés
4. Apports du Projet
5. Perspectives

Références

- Liste des ouvrages, articles, et ressources en ligne consultés, formatés selon les normes académiques.

Annexes (exemples)

- Annexe 1 : État de l'art
- Annexe 2 : Architecture microservices (ou autre)
- Annexe 3 : Script de Création de la base de données
- Annexe 4 : Extraits de codes sources
- Annexe 5 : Diagrammes supplémentaires
- Annexe 6 : Résultats des tests

Glossaire (Optionnel)

- Définitions des termes techniques utilisés dans le mémoire.

Index (Optionnel)

- Index des termes et concepts clés.